

Viva: Project 21, Dog Breed Gallery

API: Dog CEO (<https://dog.ceo/dog-api>) - Key: No key needed

Brief: Pick a dog breed and see random photos of it.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between val and var, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. How did you model the breed list response, and how did you model the images response?
4. What does ? mean on a property, and are any fields here genuinely optional?
5. Which Android component holds the selected breed, and what happens on rotation?
6. How does a GridLayoutManager differ from a LinearLayoutManager, and which did you use?
7. Where do you declare the INTERNET permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. What problem does Retrofit solve for you?
9. Explain the role of the converter factory in your Retrofit builder.
10. The breed name sits in the path. Which annotation did you use, and what is the final URL?
11. Why is your API call function marked suspend?
12. Which dispatcher runs the network call, and which updates the UI?
13. What happens if you call the API on the main thread?
14. What is a coroutine scope, and which one did you launch from?
15. How do you tell a network failure apart from a breed that does not exist?

C. The Dog CEO API and this app

16. What are the endpoints for listing all breeds and for fetching images of one breed?
17. The breed list comes back as a message object where each key is a breed and the value is a list of sub breeds. Why is that shape awkward, and how did you flatten it?
18. How do you build the URL for a sub breed like "bulldog french"? What does the path look like?
19. Every response has a status field of success or error. Do you check it before reading message?
20. The message field is sometimes a string, sometimes a list, sometimes an object. What problem does that create for a strongly typed language, and how did you solve it?
21. How many images does /images/random/50 return, and what is the maximum the API allows?
22. Which image library did you use, and why does a grid of 50 photos need caching and recycling?
23. What happens to memory if you load 50 full size images without a library?
24. Do the image URLs use HTTPS, and why does that matter for Android?

D. Edge cases, design and extensions

25. What does your app show while images are loading?
26. A user has no connection and picks a breed. Describe exactly what they see.
27. What placeholder do you show for an individual image that fails to load?
28. How would you add a full screen viewer when a photo is tapped?
29. How would you let the user save a photo to their gallery, and what permission is involved?
30. How would you cache the breed list so the picker opens instantly?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.